Bachelor's Degree Thesis

Computational Mathematics and Data Analysis

Universitat Autònoma de Barcelona

Analysis of the Internal Representations of Convolutional Neural Networks

Víctor Benito Segura

Supervised by: Maria Vanrell Martorell and Guillem Arias Bedmar

21/06/2024

Abstract

Artificial intelligence has revolutionized our society by achieving milestones that seemed impossible. At the forefront of this advance are Neural Networks, models that emulate the functioning of the human brain to learn relevant characteristics from data. Convolutional Neural Networks (CNN*), in particular, have excelled in the field of computer vision, being essential in applications such as object detection, autonomous driving or image classification.

This great success is often linked to a lack of knowledge about the internal workings of CNNs and how they achieve such impressive results. This project aims to explain the operation of these networks, through the parameterization of basic functions: Sigmoid and Gaussian, and their derivatives, focusing on the activation of neurons individually.
Analyzing which images activate the neurons of a network the most will allow us to better understand what the model is learning in each of its layers. To achieve this, a method generating simple images will be built that describe their shape and color from a set of interpretable parameters.
The Simulated Annealing optimization algorithm will then be used to search for parameters that maximize the activation of each neuron.
Finally, a set of tools for sorting and classifying the neurons of each layer of the VGG-16 network will be defined based on the parameters found with the algorithms defined above. This understanding can not only improve the design of future networks, but also increase confidence in their practical applications.

Keywords: Convolutional Neural Networks, parameters, CNN, individually neurons activation, sigmoid, gaussian, Simulated Annealing, VGG-16.

CNN*: Convolutional Neural Networks,

Índex

1 Motivation

Convolutional Neural Networks (CNNs) have achieved significant advances in a wide range of areas in recent years, from autonomous driving to human detection and clinical diagnosis. However, these impressive results of CNNs often come with a lack of understanding of their internal workings and how they manage to perform various tasks so effectively.

This work aims to explore the features that activate the network's neurons at an individual level. This approach can help provide a deeper understanding of the internal processes of CNNs and facilitate the development of future applications and related techniques.

2 Objectives

The layers of convolutional networks trained to solve visual tasks, such as object recognition, have an internal organization that makes each neuron selective to a specific feature of the images. Some neurons are activated by edges in a particular orientation, regardless of their color, while others are activated by regions of a specific color, regardless of their shape.
Examining all these properties of neurons globally in a network that has thousands of neurons is very challenging. This task could be simplified if we could automate the description of the types of neurons in a trained network, allowing for quick indexing and grouping of neurons that share certain properties.
The primary goal of this work aligns with this direction. The objective is to create an image generation system that identifies which features most activate each of the neurons in the various layers of a Neural Network, aiming for this procedure to be as automated and general as possible. Therefore, the sub-objectives proposed are as follows:

Objective 1 Develop a method to generate simple images that describe their shape and color based on a set of interpretable parameters.

Objective 2 Design an optimization algorithm that allows searching for the simple images that maximize the activation of a neuron.

Objective 3 Define a set of tools for sorting and classifying the neurons of each layer of a CNN using the parameters found with the algorithms defined in the previous objectives.

3 Basic Concepts and Related Work

Before explaining the implemented methods and presenting the obtained results, we will briefly introduce some basic concepts and review related published work. First, we will explain what a Convolutional Neural Network (CNN) is, how it is internally organized, and how it is trained. Then, we will mention some previous work that has studied the behavior of neural networks through the individual analysis of neuron activations.

3.1 Convolutional Neural Networks

Artificial Neural Networks[1], or **ANNs**, are computational processing systems inspired by the functioning of nervous systems, such as the human brain. These networks are composed of a large number of interconnected computational nodes, called neurons, that work together to process information and learn from the inputs to optimize the final output. The basic structure of an Artificial Neural Network is shown in the following figure:

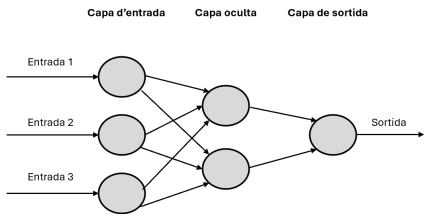

Figura 1: Example of a Basic Neural Network Structure

The main difference between CNNs and traditional ANNs is that CNNs are primarily used in the field of image pattern recognition. This is because each neuron is associated with a convolution operation using a spatial filter, which allows it to be activated when the input image contains a pattern that matches the shape

for which the filter has been trained.

To train a CNN, we need standard reference datasets for machine learning. An example is the MNIST database of handwritten digits, which has been used to train networks capable of character classification. In Figure **??**, we show an example of an MNIST image and highlight some partial patterns that are used by the network to eventually identify the number 4. CNNs have a hierarchical structure in which, through the composition of shapes at different levels, they end up identifying more complex patterns. In the deeper layers, global patterns of the images are learned, which ultimately represent all the pixels of the image. In contrast, in the shallower convolutional layers, local patterns are represented, corresponding to the size of the filter window associated with each neuron. Neurons in the intermediate layers learn to activate for local shapes that contribute to the overall identification of a character, as shown in the figure[2]:

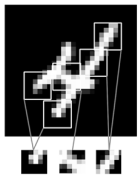

Figura 2: Example of Pattern Detection in Images on an MNIST Image.

This learning through local patterns gives convolutional networks two very interesting properties:

Translation Invariance : The features learned are invariant to translation. That is, if a CNN recognizes a pattern at the bottom of the image, it can recognize the same pattern later at a different position. This is a property derived from the convolution operation.

Hierarchical Composition : The initial layers can learn basic features such as edges, while the more advanced layers combine these features to form more diverse shapes, as shown in the following figure[2]:

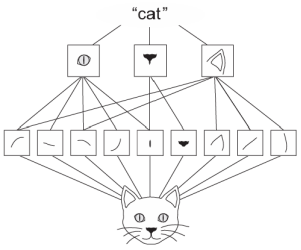

Figura 3: Example of Hierarchical Pattern Detection. Local edges are combined to detect objects such as eyes or ears, which are ultimately grouped to form high-level concepts like 'cat.'"

3.2 Example of CNN: VGG16

In this work, we will analyze the VGG-16 network [3], which emerged in 2014 as a potential improvement over the AlexNet model. VGG-16 is a very effective model, particularly appealing for its simplicity. It contains 16 layers that combine: operations with 3×3 filters, ReLU activation functions to capture non-linearity, pooling operations to reduce the number of parameters, and finally, *Fully-Connected Layers* and *Softmax* to predict the output.

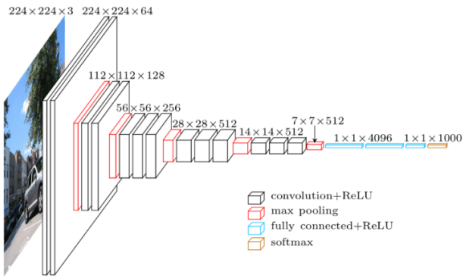

Figura 4: Diagram of the Layers of a VGG-16 Model[4]

VGG-16 achieves a 93% accuracy on the test set, using the ImageNet database, which contains over 14 million images labeled with 1,000 possible object categories.

3.3 How CNNs Are Trained: ImageNet

ImageNet[5] was created by researcher Li Fei-Fei and her team at Stanford University in 2009. Following the trend that WordNet had generated in improving natural language processing, they proposed the creation of a vast, diverse, and high-quality dataset to enhance the field of computer vision. ImageNet contains images from different classes: animals, objects, people, landscapes, etc., with very similar classes such as dog breeds, featuring images in various positions, lighting conditions, scales, and noise. It quickly became a very attractive challenge for researchers in the field, leading to the development of new models that were trained using ImageNet. This project enabled the creation of widely used models today that have contributed to the success of the field. Some of these models include Xception, VGG19, ResNet, MobileNet, DenseNet, and their derivatives, as well as the model we will use in this project: VGG-16.

3.4 Individual Neuron Analysis

As mentioned at the beginning of this work, training these CNNs with a high number of layers and neurons has yielded excellent results in many visual tasks, but the main problem is the inability to understand and predict the response of a trained network. This has opened up a research field aimed at analyzing and understanding this effectiveness. Within this area of finding applications for CNNs, one line of research has been the study of neurons individually to attempt to understand the global behavior from the comprehension of each part.
Following the MNIST example we discussed in the introduction, one way to verify the effect of an image on a specific neuron is through the concept of activation. For each neuron, there is a specific filter that scans the images to look for a particular pattern. In this image[2], we can see an example:

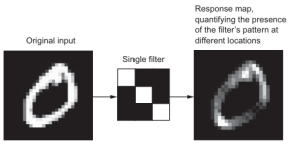

Figura 5: Response of an Image to a 3x3 Filter

In this case, we can see how the regions of the image of the zero that contain a shape similar to that of the filter are more intense in the response image.

One way to measure this intensity of the response is through activation. For each neuron, we can rank the images based on how much they activate it. This way, by observing which images activate a neuron the most, we can get an idea of what the network is learning, even without knowing the exact filters used for each neuron.

Understanding the functioning of convolutional neural networks has been a key topic within the research departments of the Computer Vision Center. So far, different studies have been conducted to clarify the learning of neurons within the network.

Key concepts:

- **Top Scoring** : Image Found That Most Activates a Specific Neuron in a Layer of the Network
- **Neuron Feature (NF)** : Average Image of the 100 Top Scoring Images That Most Activate a Specific Neuron.
- **Selectivity Index:** Index on the Importance of a Specific Feature (Color, Shape, Class) on Neuron Activation

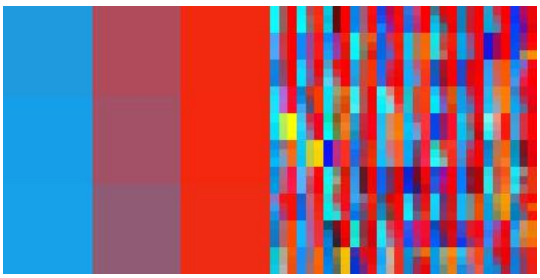

Figura 6: Example of Neuron Feature (left) and the 100 Top Scoring Images (right) for Neuron 0 in Layer 0

In this work[6], two measures of selectivity in neurons have been incorporated:
Color Selectivity: index on the importance of color to the neuron.
Class Selectivity: index on the importance of class to the neuron.
Another very interesting project [7] shows the different shapes that a neural network learns at the various layers:

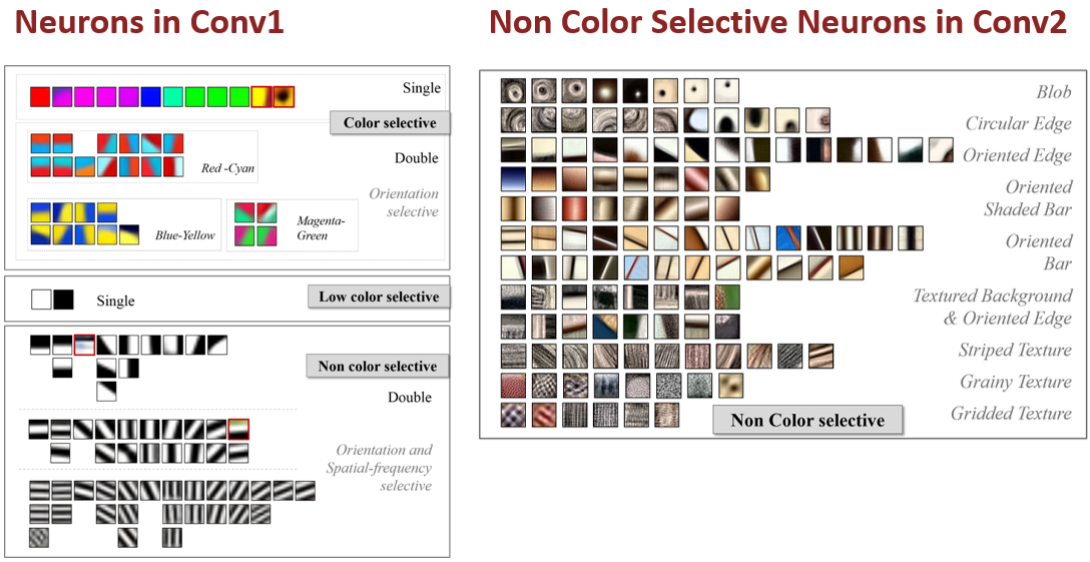

Figura 7: Ranking of Neurons in a Neural Network by Their Shape and Color Selectivity

This ranking precisely represents the motivation for our work. However, this refinement has been done manually, which is tedious and not easily generalizable to other networks. This is why the need arises to perform this same task in an automatic and general manner.

4 Generation of Simple Images

To achieve Objective 1, we need to design simple images with parametric functions that allow us to understand the activation of a neuron based on the parameters of this function.
To generate images of simple shapes, we will use two families of functions: the Sigmoid function and the Gaussian function, along with their derivatives. With these basic functions, we can generate a wide family of simple shapes where the parameters describe elements such as orientation, smoothness of contours, or spatial frequency.

4.1 Shapes Based on the Sigmoid

We will use the sigmoid function to generate linear images. That is, oriented lines in different ways, with varying contrasts and displacements.

4.1.1 1D Sigmoid

First, let's look at the definition of the basic sigmoid function in one dimension:

$$f(z) = \frac{1}{1 + e^{-z}} \qquad (1)$$

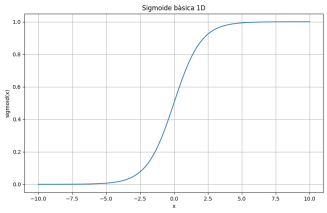

Figura 8: Basic Sigmoid Function

Next, we show the effect of the different parameters of the function. The slope, which we will call (a), and the displacement that we will define with the variable (b).

$$f(z, a, b) = \frac{1}{1 + e^{-a \cdot (z-b)}} \qquad (2)$$

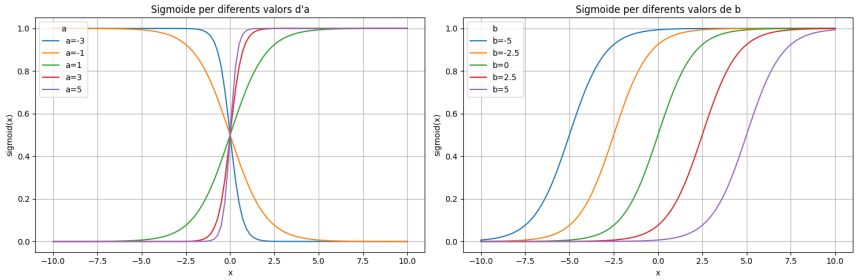

Figura 9: Different Values of Slope (a) and Displacement (b) for a 1D Sigmoid

4.1.2 2D Sigmoid

In order to create images using the sigmoid function, we will use the meshgrid function[8]. This function generates a cloud of points (x,y) within a specified range. Subsequently, we will pass this cloud of points as an argument to our two-dimensional sigmoid function to generate the resulting image.

$$f(x, y, a, b) = \frac{1}{1 + e^{-a \cdot (x+y-b)}} \qquad (3)$$

As mentioned earlier, we are interested in obtaining lines in different orientations. To achieve this, we will include a new parameter θ in the function to produce the necessary rotation.

$$f(x, y, a, b, \theta) = \frac{1}{1 + \exp(-a \cdot (rotated\_x + rotated\_y))} \qquad (4)$$

on:

$$rotated\_x = x \cdot \cos(\theta) - (y - b) \cdot \sin(\theta)$$
$$rotated\_y = x \cdot \sin(\theta) + (y - b) \cdot \cos(\theta)$$

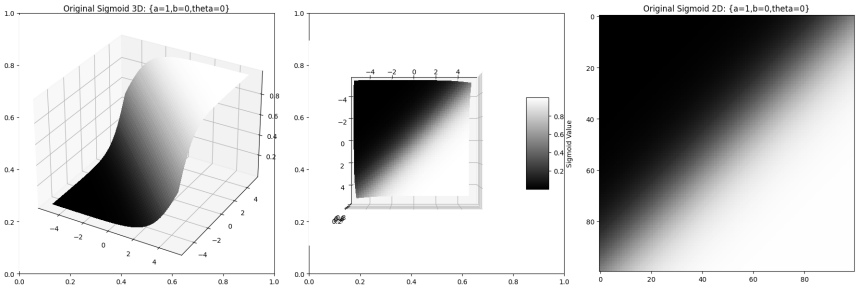

Figura 10: Generation of a 2D image with the original sigmoid function.

4.1.3 Rotation

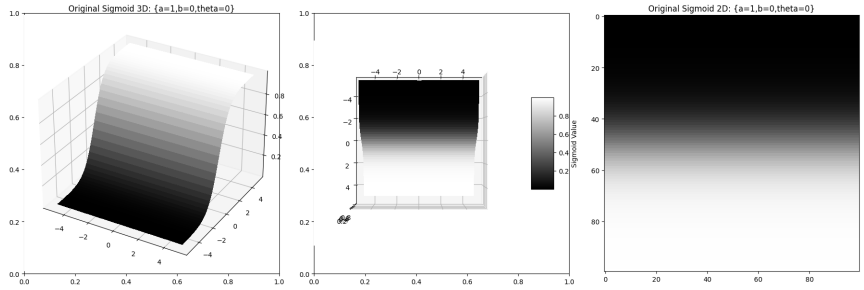

Figura 11: Generació d'una imatge 2D amb la funció sigmoide modificada

Due to the nature of the sigmoid function, the 2D projection without rotation, that is, with an angle $\theta = 0$, corresponds to an image showing a 45^{o} line 10. To correctly understand the learning of the neurons and to avoid adding ambiguity, we will define our sigmoid function as a -45^{o} rotation of the original 2D sigmoid function.

4.2 Shapes Based on the Gaussian

With sigmoids and their derivatives, we can generate a wide range of images. However, we cannot generate isotropic shapes, which is a significant drawback since we do find neurons in the network that activate for these shapes. Precisely with the aim of generating non-linear images such as circles or ellipses, we will use Gaussian functions.

4.2.1 1D Gaussian

$$g(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)} \tag{5}$$

on:

- μ is the distribution mean.
- σ is the standard deviation of the distribution.

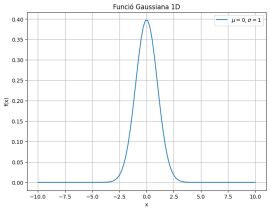

Figura 12: 1D Gaussian with $\mu = 0$ and $\sigma = 1$

Next, we present a graph to observe the behavior of the Gaussian for different values ofμ and σ

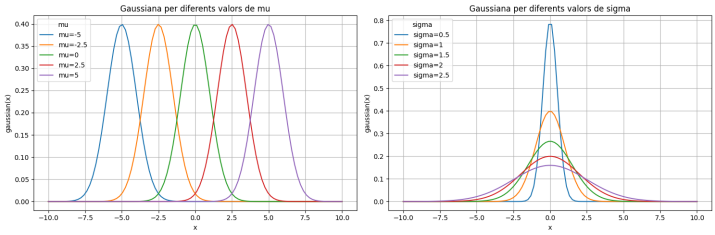

Figura 13: Different values of μ and σ for a 1D Gaussian

4.2.2 2D Gaussian

In the same way that we did with the sigmoid function, we will transform the function into two dimensions and construct the image using the cloud of points generated by the meshgrid.

$$g(x,y) = Ae^{\left(-\left(\frac{(x-\mu_x)^2}{2\sigma_x^2} + \frac{(y-\mu_y)^2}{2\sigma_y^2}\right)\right)} \tag{6}$$

where A is the amplitude.

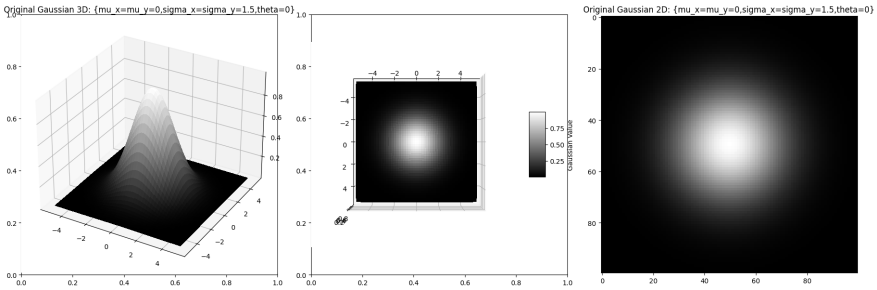

Figura 14: Generation of a 2D image with the Gaussian function

4.2.3 Rotation

Finally, we will also add rotation to the Gaussian function. To avoid ambiguity, we will define the Gaussian with a series of rules:

- In cases where $\sigma_x = \sigma_y$, we will not apply rotation.
- We will always consider that $\sigma_x > \sigma_y$ to avoid generating duplicate images.

$$g(x,y) = Ae^{-\left(\frac{(rotated_x-\mu_x)^2}{2\sigma_x^2}+\frac{(rotated_y-\mu_y)^2}{2\sigma_y^2}\right)} \tag{7}$$

on:

$$rotated_x = (x-\mu_x)\cdot\cos(\theta) - (y-\mu_y)\cdot\sin(\theta)$$
$$rotated_y = (x-\mu_x)\cdot\sin(\theta) + (y-\mu_y)\cdot\cos(\theta)$$

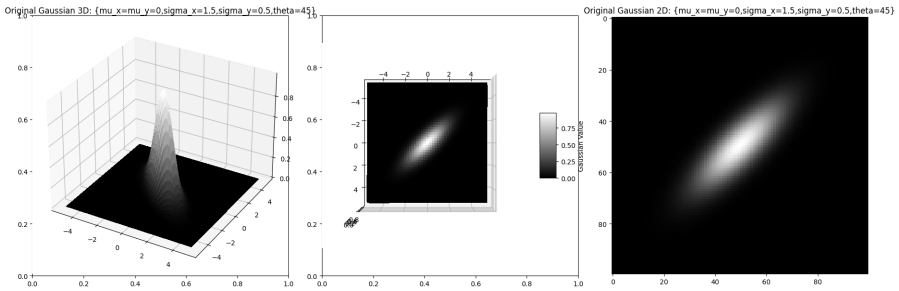

Figura 15: Generation of a 2D Image with the Gaussian Function

4.3 Calculation of Derivatives

Calculating spatial derivatives allows us to generate more complex shapes from the basic functions while maintaining the same parameters introduced. That is, new images useful for classifying images in the neural network, such as various lines or textures.

4.3.1 Sobel

To calculate these derivatives, we will use the Sobel differential operator[9], which is a widely used operator in the field of image processing to approximate the gradient of an image's intensity function.

Sobel filter in the x direction (horizontal):

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel filter in the y direction (vertical):

$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

4.3.2 Edge Effects

As we perform derivatives with the Sobel operator, edge effects occur due to how the filter sweeps across the image. This causes the outer pixels to have a high unwanted intensity, while the intensity of the central derived image decreases.

To address this problem, we will increase the pixel values of the image based on the number of derivatives we want to perform. Then, we will carry out the corresponding derivatives and finally crop the central part of the image to return a correctly derived image of the same size.

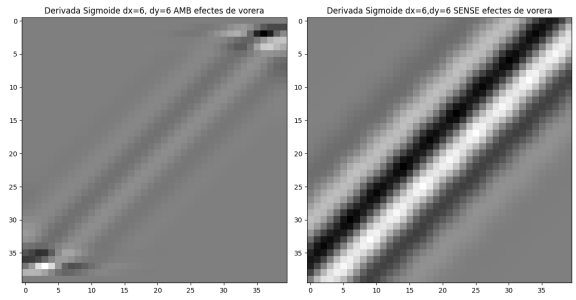

Figura 16: Visualization of the edge effects for an image generated with a sigmoid of size 40×40 with parameters $a = 1$, $b = 0$, $\theta = 45$, and derivatives $dx = 6$, $dy = 6$

5 Search for Maximum Activations

In the previous section, we proposed a method to generate images of simple shapes using basic functions. Now, we need a method to search for images that maximize the activation value of each neuron within the entire image set created. To solve this problem, we will apply a local search algorithm that explores the entire image space generated by the parameters. For each parameter set, the heuristic function to optimize will be the activation induced by the generated image on the neuron being analyzed. Therefore, for each neuron, we must apply the search algorithm across the entire parameter space.

5.1 Preliminary Computational Considerations

Initially, we considered structuring the search by generating standard parameters to initialize a local search algorithm. The algorithm created different images by modifying the parameters and saved the combination of these if the measured activation was higher than the previously obtained value. Unfortunately, this approach did not work for two main reasons:

- **Too Broad Initialization**: The different shapes that can be generated with sigmoids and gaussians have considerable variation in terms of parameters. Finding an initialization that allows us to find global maxima for such diverse images is not easy at all. Moreover, search algorithms require many iterations to reach activations close to those generated by the Top Scoring image of each neuron.
- **Parallel Iterations**: The difficulty of finding a good initialization arises from the problem of calculating activations. Each time we search for the activation of an image, we are traversing a VGG-16 network, which takes time. Furthermore, since the parameters depend on previous executions, it is not possible to execute in parallel on a GPU. Therefore, this initial approach is not viable, and we had to explore other solutions.

To address these issues, we have approached the problem from a different perspective. Since we cannot calculate activations iteratively due to computational capacity, we decided to utilize parallel execution processes to calculate a first initialization. We accomplished this by designing a two-step search algorithm:

Step 1. Grid Search : Based on generating a set of images with a set of parameters sampled exhaustively over the parameter space as if we were projecting a grid. From these images, we will calculate the executions in parallel using GPUs. Then, we will use an argmax function to find the parameters that produce the best initializations.

Step 2. Iterative Search : Once we have found a good initialization, we will apply a local search algorithm around the parameters of the images with the best activations. There are several alternatives. In this first approach, we have decided to work with *Simulated Annealing*.

5.2 Step 1: Grid Search

In this first step, we need to generate a set of initial images with a reduced range of parameter values of the functions. The goal is to produce sufficiently good initial approximations without overloading the GPU memory. This is possible because these parameters are independent, and we can compute the activations of all these images at once, in a single pass of the VGG-16 for each layer. For each neuron in each layer, we can find the images that most activate that neuron and save the parameters that generate them. We will build as many subsets of images as there are layers in our neural network. The size of the generated images in the different subsets is determined by the size K of the images for that layer.

5.2.1 Shape Parameters

We will generate an initial set of images consisting of different basic shapes from the sigmoid and Gaussian functions. These images will be generated from the range of possible values that each of the function parameters can take, which we summarize in the following tables. We will also illustrate the behavior of each parameter with the following grids of images.

Taula 1: Sigmoid

Parameter	**symbol**	**Grid Image values**
Slope	a	[0.1, 1.55, 3]
Shift	b	[-3, -1.8, -0.6, 0.6, 1.8, 3]
Angle	θ	[0, 30, 60, 90, 120, 150, 180, 210, 240, 270, 300, 330]
X Derivative	dx	[0, 1, 2, 3, 4]
Y Derivative	dy	[0, 1, 2, 3, 4]

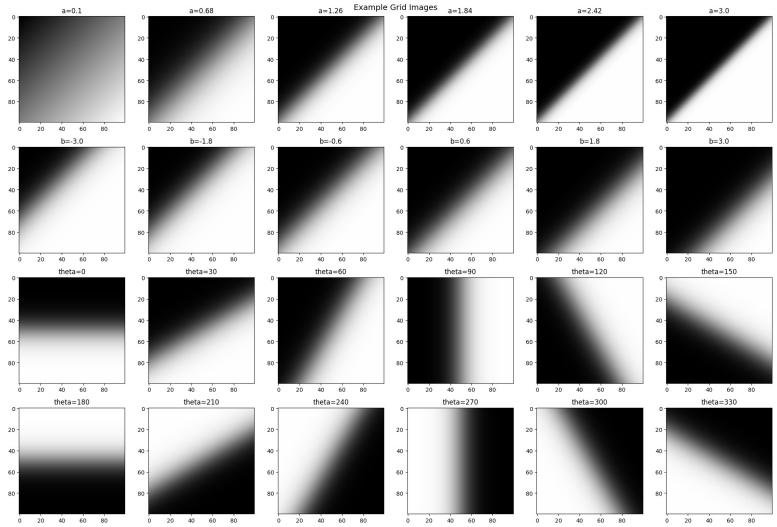

Figura 17: Example of Grid Images based on a Sigmoid Function with parameters $a = 1, b = 0, \theta = 45, dx = 0, dy = 0$

Taula 2: Gaussian

Parameter	**Symbol**	**Grid Image Values**
Center of the Gaussian on the x-axis.	μ_x	[-5, -2.5, 0, 2.5, 5]
Center of the Gaussian on the y-axis.	μ_y	[-5, -2.5, 0, 2.5, 5]
Standard Deviation x-axis	σ_x	[0.10, 0.48, 0.86, 1.24, 1.62, 2.00]
Standard Deviation y-axis	σ_y	[0.10, 0.48, 0.86, 1.24, 1.62, 2.00]
X Derivative	dx	[0, 1, 2, 3, 4]
Y Derivative	dy	[0, 1, 2, 3, 4]

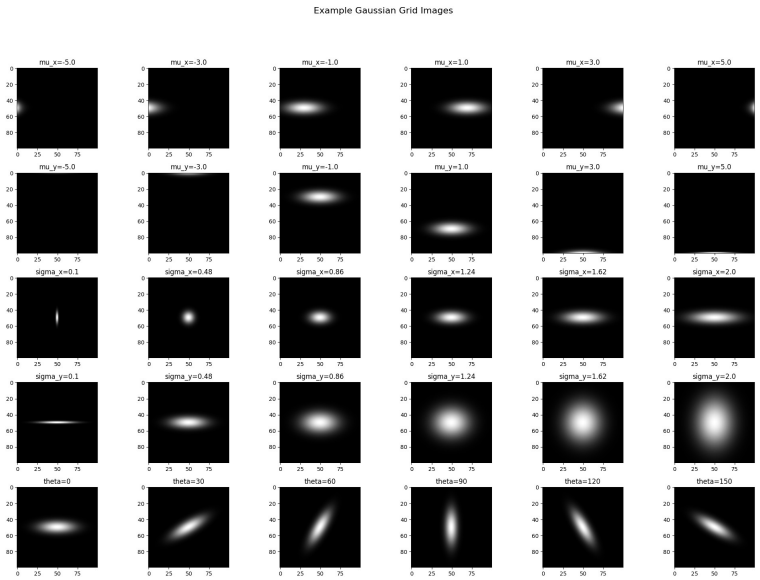

Figura 18: Example of Grid Images based on a Gaussian Function with parameters $\mu_x ==, b = 0, \theta = 45, dx = 0, dy = 0$

Synonyms and Hash Functions

The images generated with sigmoid and Gaussian functions, along with the previously mentioned parameters, often result in identical images. This is due to the symmetries of the functions. Specifically, this occurs in the first layers because the parameter range is smaller.
In order to avoid generating identical images, which increase dimensionality and produce the same neuron activations, we will use Hash functions[10]. Hash functions are a widely used tool in Cryptography for encrypting and decrypting messages. In this project, we will use them to find those parameters that generate the same images. We will look for strong collisions between the hash values produced by the matrix values of the images. This will serve two main purposes:

- First, we will be able to significantly reduce the image space.
- Secondly, it will help us better understand the relationships between the parameters and determine which sets of parameters produce the same images.

5.2.2 Color Selectivity

As we observed in previous studies, some neurons are highly selective to color, while others are less so and are more selective to other features such as shape.
Our initial set of images contains black and white images, which often do not sufficiently activate certain neurons. Therefore, by leveraging the color selectivity index, we will add color to the images of the neurons that exceed a predefined threshold.

5.2.3 Color Parameters

To colorize the images, we will use the images that have most strongly activated a neuron (Top Scoring). Next, we will use a clustering algorithm, KMeans[11], to find the two most prominent colors in the image. We will then perform a transformation of the grayscale of our black-and-white images into a scale that starts from color 1, found with KMeans, and transitions to color 2. The following figure illustrates how we transform the images in our set to color using the representative colors of the neuron's Top Scoring image.

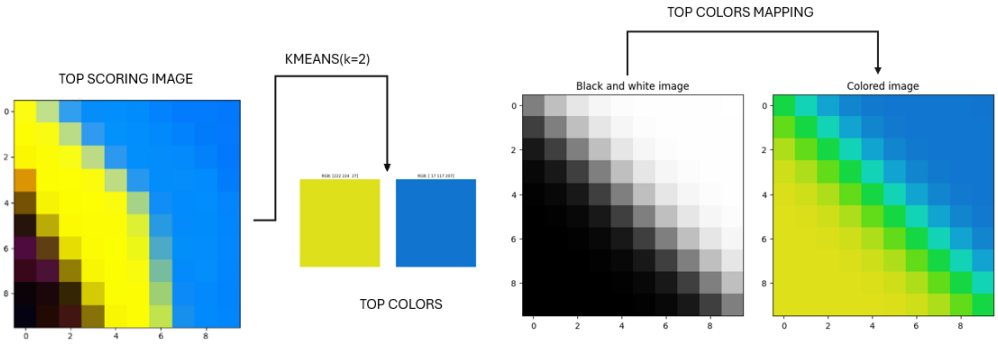

Figura 19: Example of the process of colorizing a black-and-white image using the representative colors obtained with KMeans from the Top Scoring image.

After transforming the images to color for the selective neurons, we have a set of images for each neuron from which we can calculate the activations. For each neuron, we will display the Top Scoring image and the five images found within our image set that have produced the highest activation:

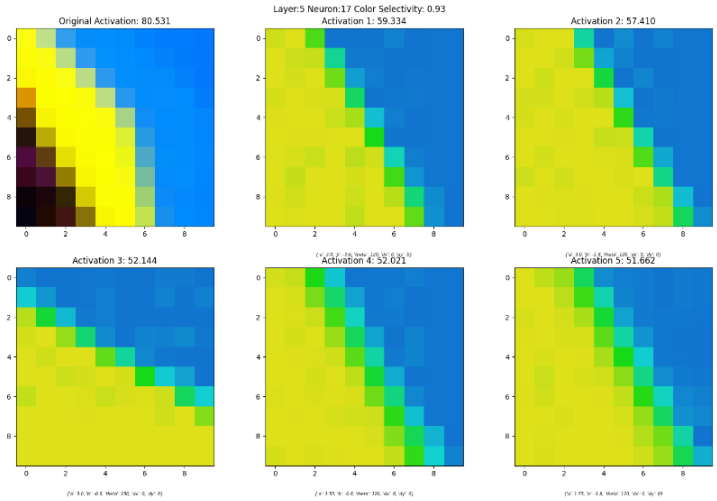

Figura 20: Example of the activation results obtained from the image set in neuron 17 of layer 2.

5.3 Step 2: Iterative Search

At this point, for each neuron, we have five images, either colored or not, that approximate the maximum activation of the neuron produced by the Top Scoring image. However, these images used are a limited subset of the parameter values that produce the images. That's why, once these nearby images are found, we will use a local search algorithm to further optimize the activation value.

5.3.1 Simulated Annealing

Simulated Annealing [12] is a local search algorithm used to solve optimization problems. The strength of the algorithm lies in its ability to escape local optimal values by allowing uphill moves. In other words, it makes moves that worsen the objective function's value with the hope of finding a global optimal value later.

The algorithm is named after the physical process of annealing solids, where they are heated and then slowly cooled to reach an optimal configuration. This process is similar to searching for global minima in discrete optimization problems. The algorithm always accepts solutions that improve the current one and occasionally accepts worse solutions to avoid getting stuck in local minima. The probability of accepting inferior solutions depends on a temperature parameter that decreases with each iteration. Initially, more errors are allowed to explore different paths, and as the temperature decreases, the algorithm tends to focus on global optimal solutions.

We will initialize five Simulated Annealings, taking as initial parameters the parameters of the five images

from our image set that most activated the neuron. Additionally, we will include the RGB parameters of the colors found for the color-selective images. Finally, we will retain the parameters of the image that most activates the neuron from these five searches.

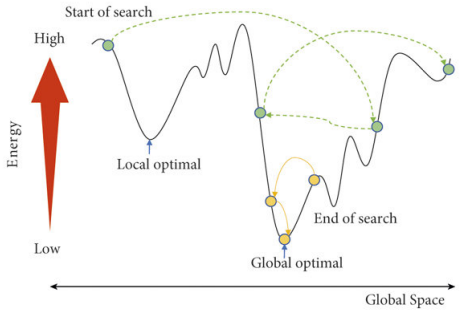

Figura 21: Example of Global Maximum Search with Simulated Annealing[13]

Finally, we can observe the results of the optimization process with Simulated Annealing for the same neuron as in Figure 20:

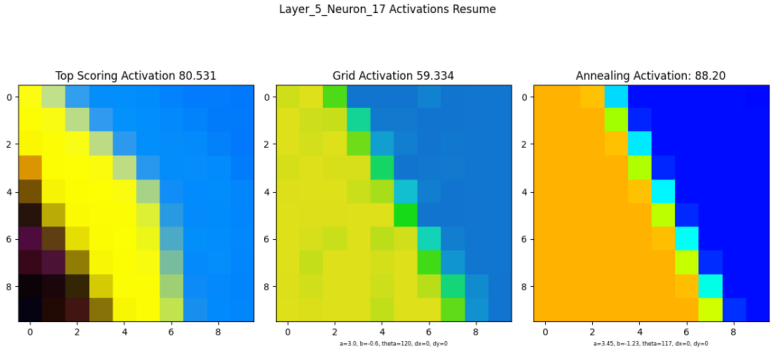

Figura 22: Exemple dels resultats d'optimització obtinguts amb Simulated Annealing a la neurona 17 de la capa 2

5.4 Optimization Results

To evaluate the optimization process of the search algorithm, we will present the different results obtained:

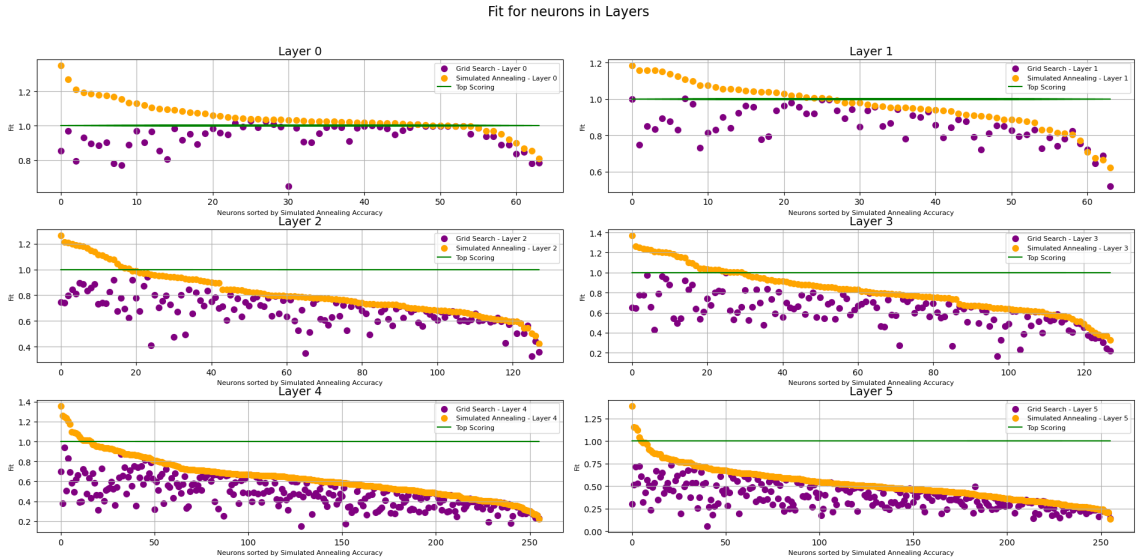

Figura 23: Adjustment of the neurons for the first six layers. In green, the target adjustment of the Top Scoring; in purple, the adjustment obtained with the image set; and in yellow, the adjustment obtained with the optimization algorithm

In this first graph, we can see how the optimization algorithm performs brilliantly in the first two layers. The adjustment of the images obtained with the optimization method, shown in yellow in the graph, surpasses the best activation obtained with the network images (Top Scoring) in most neurons.
Similarly, we can observe that as we delve deeper into the network, we generate a poorer adjustment since we are unable to represent more complex images.
Next, we will present the average adjustment results per layer, which can help us better summarize the findings.

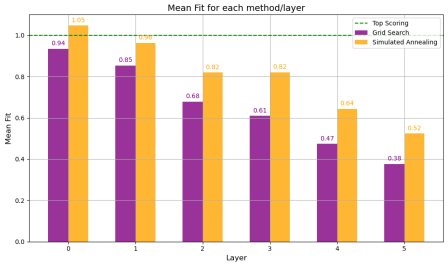

Figura 24: Average adjustment of the neurons in the first six layers of the network, for the different proposed methods.

As we had already intuited from the previous graph, the algorithm works perfectly for the first layers, but as we increase complexity and the dimensions of the images, the results become less favorable.
Seeing these results, we may wonder why we produce better activations than the images from the network. This fact can be better explained by the following table:

Taula 3: Average adjustment per Layer

Layer	Average Adjustment of Non-Color Selective Neurons	Average Adjustment of Color Selective Neu
0	0.9800	1.0903
1	0.8950	1.0510
2	0.7694	0.9536
3	0.7354	0.9111
4	0.5776	0.6992
5	0.4500	0.5713

As we can observe, the activations of the neurons we have treated with color—meaning those that have a high color selectivity—generate higher average activations than the neurons that are less selective to color. This phenomenon is due to the inclusion of the RGB values of the two main colors of the image as parameters to

optimize. By doing this, we can generate many more combinations of a color than those that appear in the network, achieving higher activations for certain neurons where color selectivity is high.

Satisfactory Results: Below, we will show some examples where the optimization algorithm has performed surprisingly well:

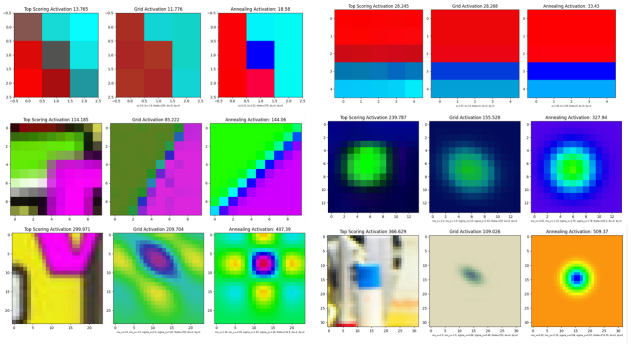

Figura 25: Example of a successful optimization case for each of the first six layers of the network

In all these cases, from different layers, we have been able to achieve an activation higher than that established with the images from the network. We can observe how in these neurons the colors are more intense than in the original images.

Unfavorable Results: Similarly to what we did for the favorable cases, we will show one neuron from each layer that we have not been able to adjust correctly in order to analyze what the errors were:

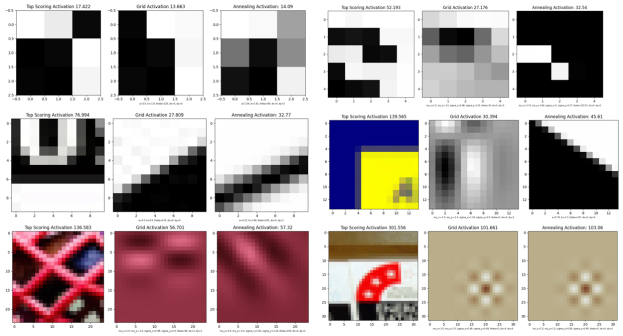

Figura 26: Example of an Unfavorable Optimization Case for Each of the First Six Layers of the Network

Based on these results, we can begin to describe the weaknesses of the algorithm:

- First of all, the algorithm is unable to properly adjust the shape for **complex images**, as we can observe in the different neurons.
- Furthermore, in certain neurons, we hardly manage to improve the activation obtained with the Grid images through optimization. This occurs when the best images in our set are very similar, and we **fail to escape the local maximum found**, as we can see in the following figure.

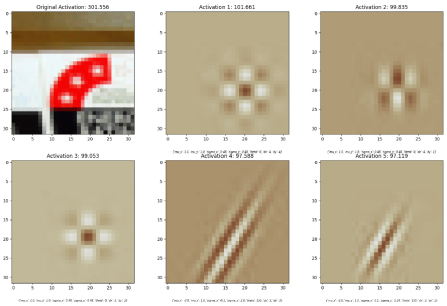

Figura 27: Example of an Unfavorable Case Where the Best Images Found in Our Image Set Are Similar.

6 Classification and Sorting of Neurons

Once the images that most activate the neurons of each layer have been optimized, we are interested in taking a general look at what the network has learned, as we want to know what shapes are being processed at each stage.
The established parameters allow us to sort the neurons based on the patterns they are internalizing. For example, we can sort the images by angles to observe the contours that are being sought, or by the number of derivatives. Since we have the parameters of the functions generated with simple functions and these fit the images processed by the network, we can create a sorting of the representative images of each neuron in each layer (NF) based on our parameters.

First, we will take a detailed view of what is being learned at each stage. We will check that our images correctly fit the Neuron Feature by displaying a graph with all the NFs of the neurons in the layers, along with our generated image 34.
We observe how in the initial layers, the network learns to detect contours and subsequently basic isotropic shapes. As we delve into the deeper layers, the shapes become complex objects, which makes our image predictions insufficiently fit.

However, this way of observing the layers is very general and does not allow us to clearly see which shapes are being addressed and which are not, a very important aspect if we want to improve the performance of a specific neural network.

6.0.1 Classification Method:

We will classify the neurons in four different ways:

- **By Layer:** First, we will separate the neurons by layers to see what the network is learning at each stage.
- **By Generating Function:** Secondly, we will divide the images based on whether they have been generated with sigmoid or Gaussian functions.
- **By Color Selectivity:** Next, we will separate the images according to whether they are color selective or not. It is worth noting that we have used a selectivity index threshold of 0.2.
- **By Derivatives:** Finally, we will divide the images based on whether they come from a derivative function or not.

6.0.2 Ordering Method:

Finally, we will order each subset of images based on the angle parameter. This will help us visualize the shapes that are understood and also those that are absent.

Refinement: As we have noted in the results of the optimization algorithms, we often generate images that do not represent the same shape as the Neuron Feature (NF), especially in the deeper layers. If the image approximation is poor, it makes no sense to use its parameters to describe the NF, as they will not define the same shape. Therefore, for this analysis, we will only show the images of the neurons that have obtained a fit superior to a certain threshold. We will slightly lower this threshold for deeper layers to be able to observe more neurons in detail.

6.1 Results of Neuron Ordering and Classification

Layer 0: We used a threshold of 0.8 and successfully fitted 64 out of the 64 images of the layer.

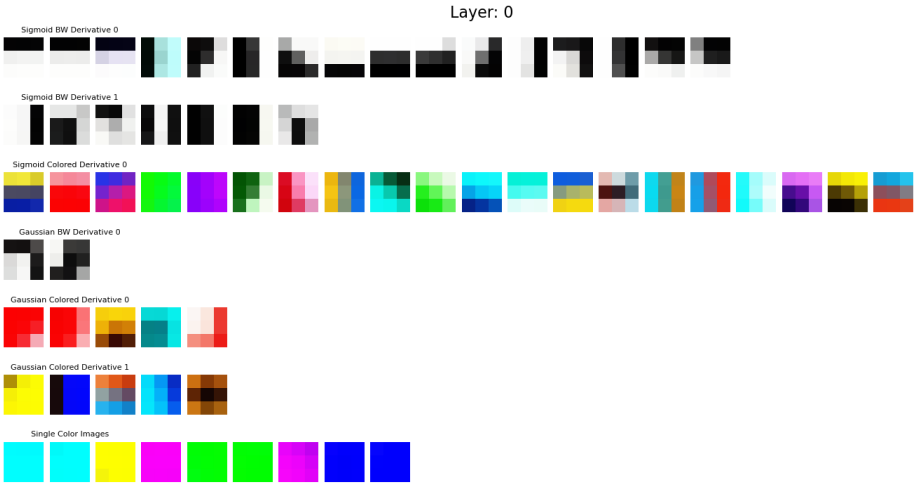

Figura 28: Classification and Ordering of Layer 0

Now we can clearly observe what each neuron is learning. We can see how in this first layer, VGG-16 learns basic contours oriented in different ways. On the other hand, we can also confirm how some neurons are responsible for processing basic colors in this layer. Finally, we can see how some derivatives and more complex shapes start to appear.

Layer 1: We have used a threshold of 0.8 and have correctly adjusted 59 of the 64 images in the layer.

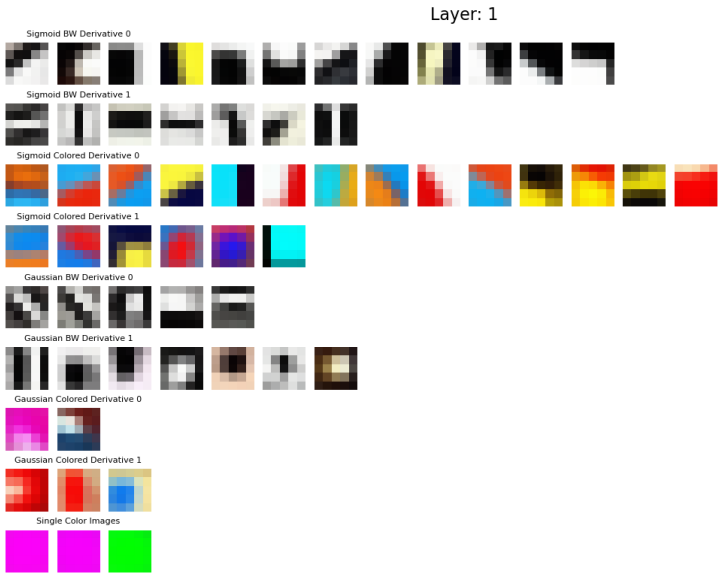

Figura 29: Classification and Ordering of Layer 1

We quickly observe how the neurons in the second layer continue a similar learning process to those in the first layer: processing oriented contours and simple colors. However, in this layer, we can see how isotropic shapes and some higher derivatives begin to appear.

Layer 2: We have used a threshold of 0.775 and have correctly adjusted 70 of the 128 images in the layer.

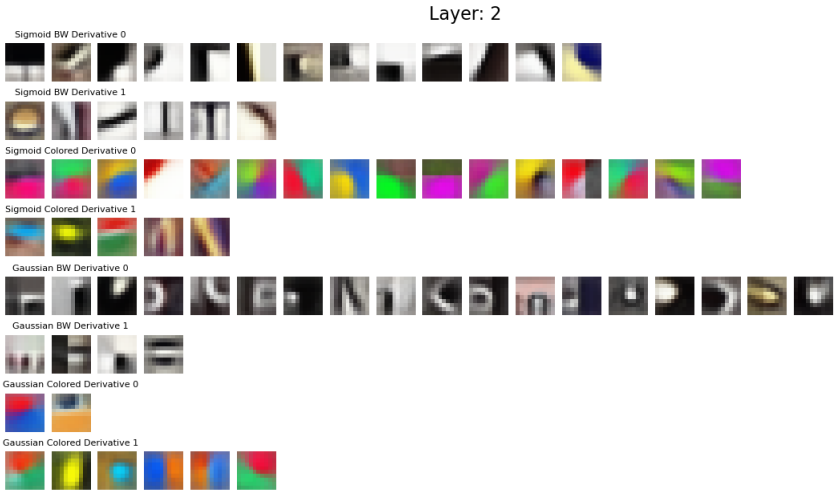

Figura 30: Classification and Ordering of Layer 2

In this third layer, we can clearly see how non-linear images begin to appear. On the other hand, the number of neurons processing shapes similar to Gaussians increases significantly. We can also visualize how shifted and rotated non-isotropic Gaussians emerge.

Layer 3: We have used a threshold of 0.775 and have correctly adjusted 72 of the 128 images in the layer.

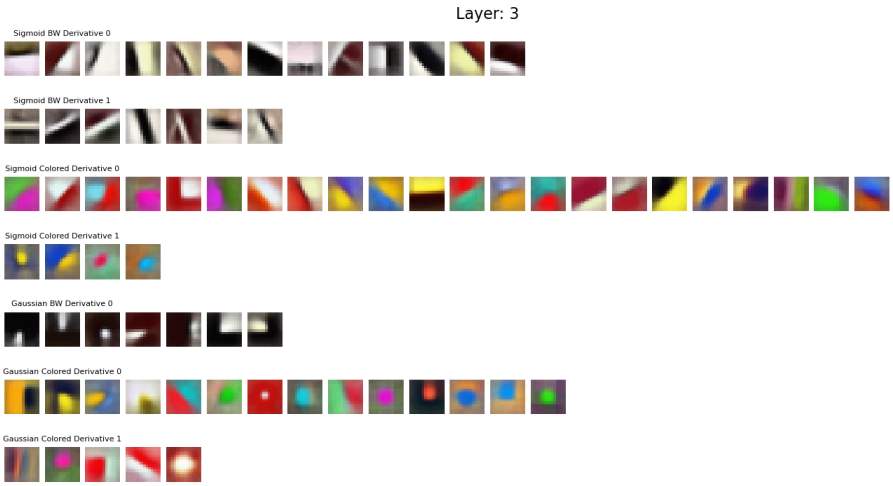

Figura 31: Classification and Ordering of Layer 3

From this fourth layer, we cannot draw many significant conclusions. The learning patterns of the neurons in the third layer continue. However, we can observe more variety in the derived Gaussians.

Layer 4: We have used a threshold of 0.75 and have correctly adjusted 63 of the 256 images in the layer.

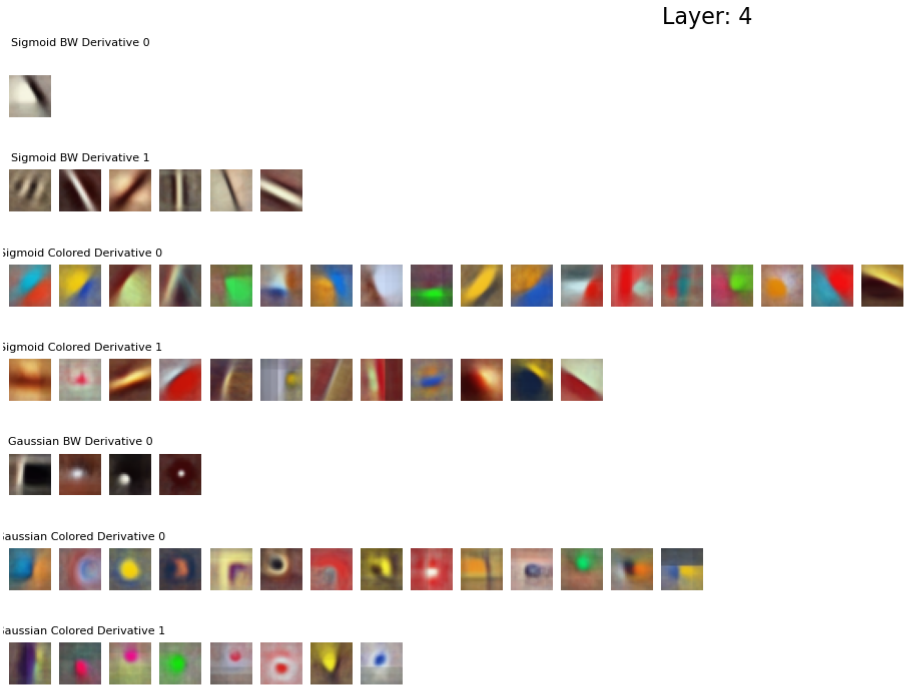

Figura 32: Classification and Ordering of Layer 4

In the fifth layer, we can observe a significant increase in the number of neurons described by derivative functions. We also find that the neurons we successfully adjust are mostly isotropic Gaussians and complex orientations.

Layer 5: We have used a threshold of 0.70 and have correctly adjusted 41 of the 256 images in the layer.

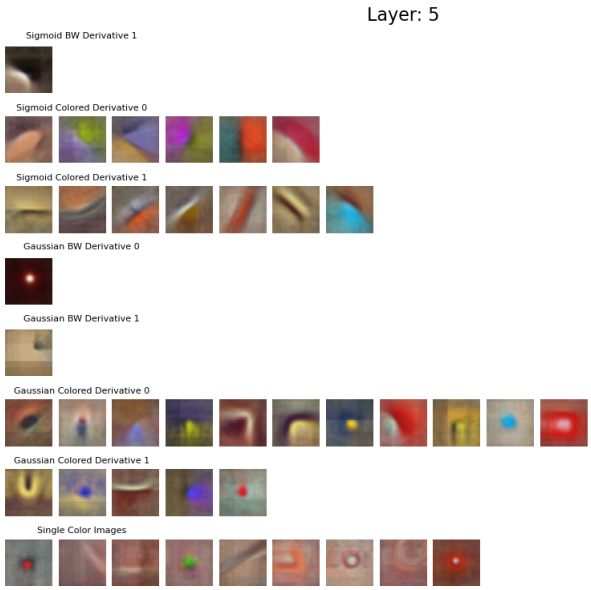

Figura 33: Classification and Ordering of Layer 5

Finally, we will analyze the last processed layer. We observe that the neurons in these layers no longer process simple contours but rather nonlinear contours. Moreover, the neurons begin to describe more complex objects that we cannot adequately adjust with our basic functions.

Conclusions from the Layer Analysis: This analysis has allowed us to clearly understand what the neurons of the different layers of the network learn. We have confirmed that the neurons in the earlier layers learn basic shapes. This learning operates hierarchically, such that the network processes increasingly complex shapes until it can comprehend objects..

Regarding our sorting method, we have seen that in the earlier layers, we can correctly adjust the vast majority of the neurons. In contrast, this is not the case in the deeper layers. This observation has led us to establish the adjustment value as a hyperparameter: a very stringent adjustment value will eliminate many neurons that are correctly adjusting patterns, while a very low value will inaccurately describe neurons.

Despite the mentioned problems, this tool can be a valuable approach for analyzing different neural networks and allows for significant room for improvement.

7 Conclusions and Future Directions

8 Conclusions and Future Directions

In this project, we have developed an initial approach to the problem of automatically characterizing the basic shapes that activate the neurons of a Convolutional Neural Network (CNN). This approach was implemented through the generation of images using parameters from sigmoid and Gaussian functions, along with their derivatives. Subsequently, we proposed a two-step search algorithm: the creation of an image space through a range of parameters and the optimization of these parameters using the local search algorithm Simulated Annealing. This strategy takes advantage of the parallel execution capabilities of CNNs to optimize the individual activation of each neuron.
Furthermore, we employed hash functions to eliminate synonyms within the parameter space and proposed a method for handling color, which presents numerous opportunities for improvement. Finally, we analyzed the results obtained and introduced a method for the automatic classification and ordering of neurons in a network, which can enhance understanding.

This work has served as a preliminary approach to the problem, opening up several new research avenues to tackle the issue, which we will summarize below:

- **Utilizing Derivative Functions:** Employing derivative functions of sigmoid and Gaussian functions, such as addition or multiplication, to address more complex shapes.
- **Threshold Parameter Study:** Investigating the threshold hyperparameter for color selectivity to improve activations without significantly increasing computation time.
- **Diverse Initial Parameters:** Ensuring that the initial parameters of Simulated Annealing are sufficiently diverse to avoid getting stuck in local maxima.
- **Extended Simulated Annealing Iterations:** Executing Simulated Annealing for more iterations with a higher temperature to avoid falling into local minima.
- **Hyperparameter Adjustment Threshold:** Studying the threshold hyperparameter for adjustment to classify and order a sufficiently large set of representative neurons.
- **Testing with Other CNN Architectures:** Evaluating the method's performance on other Convolutional Neural Networks such as Clic or VGG-19.

9 Acknowledgments

I would like to thank Maria and Guillem, the tutors of this project, for all their support during the project and for introducing me to the world of research in such an exciting field as neural networks.
Finally, I would also like to thank the Computer Vision Center of Catalonia for allowing me to use their computing tools; without these resources, it would not have been possible to complete this work.

Referències

[1] Keiron O'shea i Ryan Nash. "An introduction to convolutional neural networks". A: arXiv preprint arXiv:1511.08458 (2015).

[2] Chollet Francois. Deep learning with Python. 2018.

[3] Karen Simonyan i Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". A: arXiv preprint arXiv:1409.1556 (2014).

[4] Will Nash, Tom Drummond i Nick Birbilis. "A review of deep learning in the study of materials degradation". A: npj Materials Degradation 2.1 (2018), pàg. 37.

[5] Jia Deng et al. "Imagenet: A large-scale hierarchical image database". A: 2009 IEEE conference on computer vision and Ieee. 2009, pàg. 248 - 255.

[6] Ivet Rafegas et al. "Understanding trained CNNs by indexing neuron selectivity". A: Pattern Recognition Letters 136 (2020), pàg. 318 - 325.

[7] Ivet Rafegas i Maria Vanrell. "Color encoding in biologically-inspired convolutional neural networks". A: Vision research 151 (2018), pàg. 7 - 17.

[8] Charles R Harris et al. "Array programming with NumPy". A: Nature 585.7825 (2020), pàg. 357 - 362.

[9] Irwin Sobel i Gary Feldman. A 3x3 isotropic gradient operator for image processing. Inf. tèc. 671. Stanford University Artificial Intelligence Project (SAIL), 1968.

[10] Fips Pub. "Secure hash standard (shs)". A: Fips pub 180.4 (2012).

[11] Stuart P Lloyd. "Least squares quantization in PCM". A: IEEE transactions on information theory 28.2 (1982), pàg. 129 - 137.

[12] Alexander G Nikolaev i Sheldon H Jacobson. "Simulated annealing". A: Handbook of metaheuristics (2010), pàg. 1 - 39.

[13] Xiaoshan Yang i Weiwei Guan. "Research on logistics distribution route optimization based on deep learning model and block chain technology". A: 3c Empresa: investigación y pensamiento crítico 12.1 (2023), pàg. 68 - 85.

10 Appendix

10.1 Comparison of Generated Images vs. Neuron Features

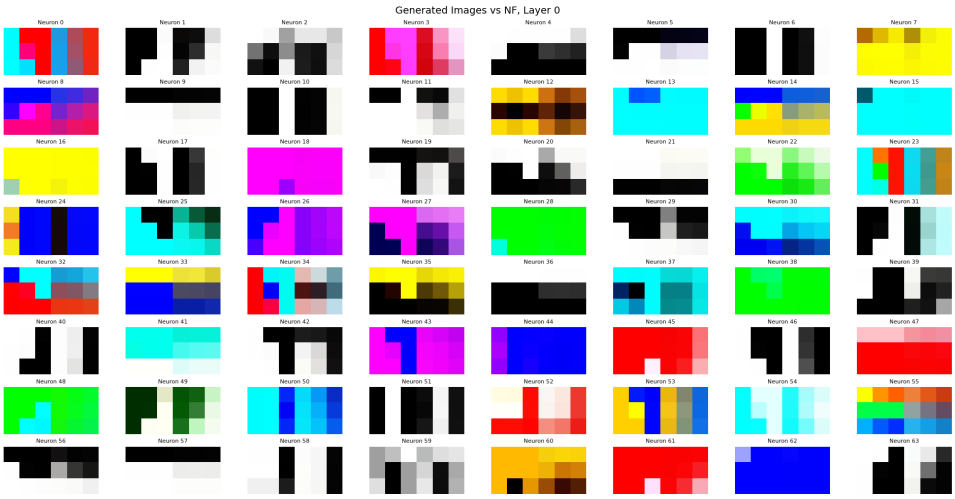

Figura 34: Comparison of Generated Images vs NF Layer 0

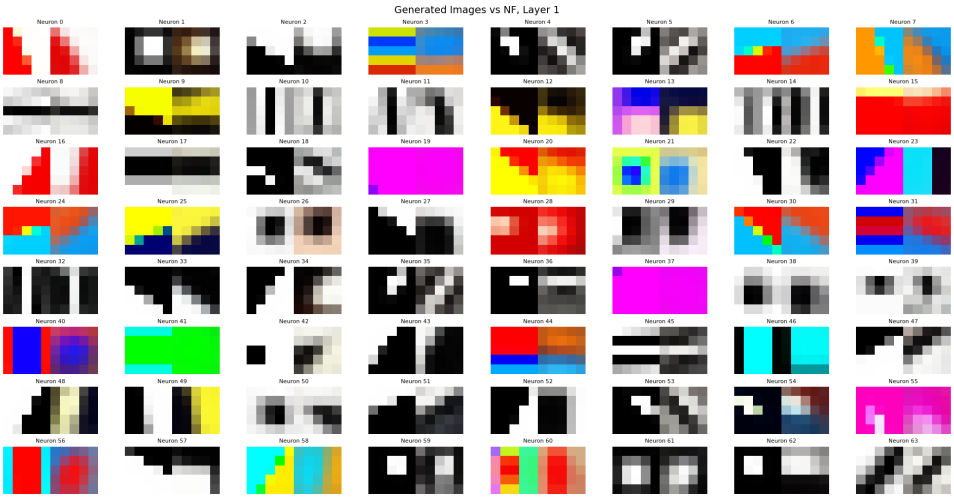

Figura 35: Comparison of Generated Images vs NF Layer 1

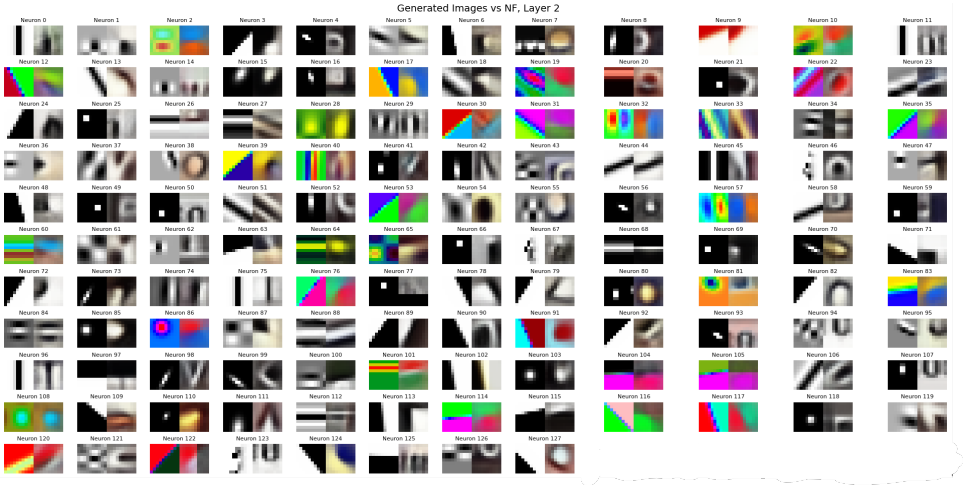

Figura 36: Comparison of Generated Images vs NF Layer 2

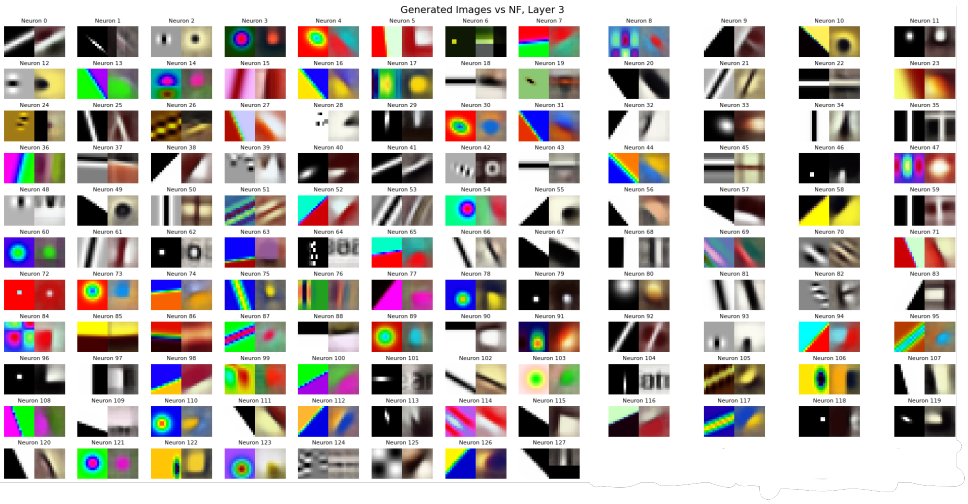

Figura 37: Comparison of Generated Images vs NF Layer 3

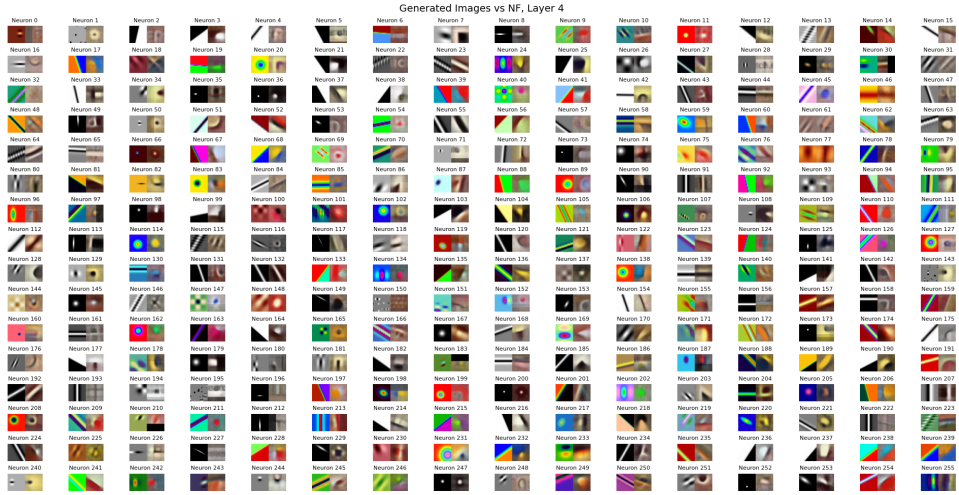

Figura 38: Comparison of Generated Images vs NF Layer 4

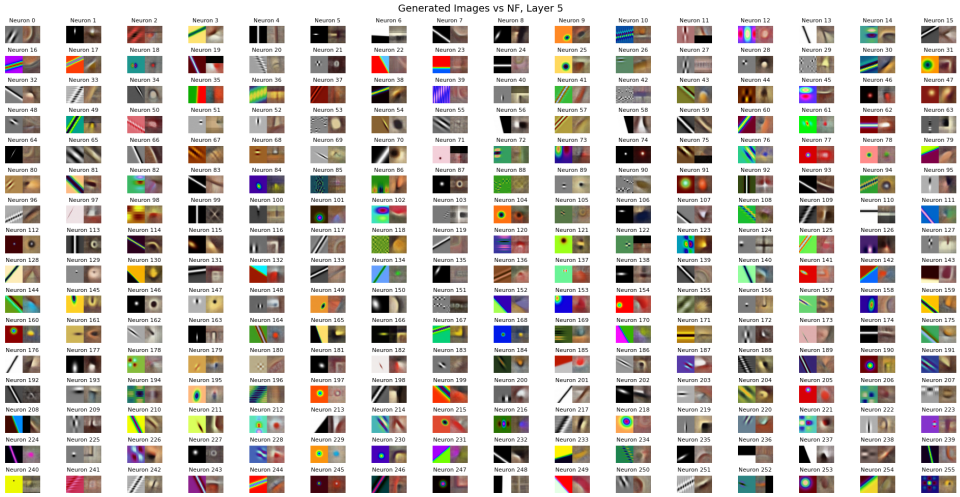

Figura 39: Comparison of Generated Images vs NF Layer 5

10.2 Project Code Link

Link to the GitHub repository with the project code:
https://github.com/Victorbs18/Understanding-Convolutional-Neural-Networks